

Writing Paper's Abstract and Introduction

Vu Nguyen

University of Science, VNU-HCMC

Content adapted from:

- Tao Xie
- Simon Peyton Jones
- Kurt Stirewalt
- Standord's InfoLab

Outline

- Tools
- Paper organization
- Title
- Abstract
- Introduction

Tools for writing and managing papers

Purpose	Tools
Write paper collaboratively	Overleaf , Office 365, Google Docs
Manage and annotate citations and papers	Mendeley , Zotero, Papers, JabRef
Search papers and more	Google Scholar , Semantic Scholar, Research Rabbit
Analyze papers, chat with PDF	Elicit , ChatGPT , SCISPACE (typeset.io), Research Rabbit, ChatPDF
Revise papers	ChatGPT , Grammarly, Trinkai.ai, Gemini
Diagrams, brainstorming	Miro , XMind, draw.io
Notes, documents, wiki	Notion, Google Docs, Obsidian
Other AI tools	askyourpdf.com , ChatPDF, cowriter.ai

Typical Paper Organization

- Title, along with names, affiliations, email addresses of authors
- Abstract
- Introduction
- Background
- Related work
- Proposed approach
- Evaluation
 - Experimental design
 - Results
 - Discussions
- Threats to validity (optional)
- Conclusions

Title – 1

- Title is very important for a paper
 - It is the first thing people notice about your paper
 - It is the title that people decide to read your paper
- It contains keywords, usually at the beginning
- If possible, name your method in the title
 - “SRD: a smart reader of human emotions”
- Avoid using uncommon buzzwords in the title
 - Readers may not understand them
- Be specific enough but not too specific about your techniques and technical terms/jargons

Title – 2

■ Titles should be concise and informative

- ✗ □ “A New Signature File Scheme Based on Multiple-Block Descriptor Files for Indexing Very Large Data Bases”.
- ✓ □ “Signature File Indexes Based on Multiple-Block Descriptor Files”

- ✗ □ “An Investigation of the Effectiveness of Extensions to Standard Ranking Techniques for Large Text Collections”
- ✓ □ “Extensions to Ranking Techniques for Large Text Collections”

■ Titles do not need to be complete sentences

- ✗ □ “Duplication of Data Leads to Reduction in Network Traffic”
- ✓ □ “Duplication of Data to Reduce Network Traffic”

Source: “[How to write a great research paper - Seven simple suggestions](#)” by Simon Peyton Jones

Title – 3

■ Some reviewers don't like your claimed approach to be “novel”

✗ □ “TestEra: A Novel Framework for Automated Testing of Java Programs”

✓ □ “TestEra: Specification-based Testing of Java Programs Using SAT”

Abstract

- Abstract is the second most read section of a paper
 - The most important part of the paper
 - It is read by 10 to 500 times more than the paper
- It helps readers to decide whether the paper is relevant to them
 - Readers also decide whether to read further
- Opportunity and risk
 - Good writing entices further reading
 - Bad writing discourages readers (and search engines) even if your ideas are good
- Readers reading the abstract is more likely interested in your paper than title readers

Abstract's Structure - 1

- Short common abstract: 50-200 words, 4-10 sentences
- Extended abstract: up to 500 words
- Abstract is a short summary of a paper, and it typically includes the following elements
 - Problem: what is the problem/context you try to address in the paper?
 - Method: how do you address the problem, your approach to solving the problem
 - Results: what are the most important results people should know?
 - Conclusions: what do you want people to take from your solution?

Abstract's Structure - 2

- Some conferences and many journals require a “structured abstract”
 - For example, Journal of Software and Information Technology uses the following headings, each starting a separated paragraph
 - Context, Objective, Method, Results, Conclusions

Context: A software estimation model is often built using historical project data. As software development practices change over time, however, a model based on past data may not make accurate predictions for a new project.

Objectives: We investigate the use of moving windows to determine relevant training data for COCOMO calibration.

Method: We present a windowing calibration approach to calibrating COCOMO and assess performance of effort estimation models calibrated using windows and all data.

Results: Our results show that calibrating COCOMO using small windows of the most recently completed projects generates superior estimates than using all available historical projects. Large windows tend to produce worse estimates.

Conclusions: This study provides empirical evidence to support the use of small windows of projects completed so far to calibrate models when COCOMO-like data is available. Additionally, when the change in software development over time is rapid, the use of windows is more justifiable for improving estimation accuracy.

Abstract's Structure - 3

■ Another example

When projects lack sufficient local data to make predictions, they try to transfer information from other projects. How can we best support this process? In the field of software engineering, transfer learning has been shown to be effective for defect prediction. This paper checks whether it is possible to build transfer learners for software effort estimation. We use data on 154 projects from 2 sources to investigate transfer learning between different time intervals and 195 projects from 51 sources to provide evidence on the value of transfer learning for traditional cross-company learning problems. We find that the same transfer learning method can be useful for transfer effort estimation results for the cross-company learning problem and the cross-time learning problem. It is misguided to think that: (1) Old data of an organization is irrelevant to current context or (2) data of another organization cannot be used for local solutions. Transfer learning is a promising research direction that transfers relevant cross data between time intervals and domains.

Kocaguneli, E., Menzies, T., & Mendes, E. (2015). Transfer learning in effort estimation. *Empirical Software Engineering*, 20(3), 813-843.

Abstract's Structure - 4

■ Another example

Testing in Continuous Integration (CI) involves test case prioritization, selection, and execution at each cycle. Selecting the most promising test cases to detect bugs is hard if there are uncertainties on the impact of committed code changes or, if traceability links between code and tests are not available. This paper introduces RETECTS, a new method for automatically learning test case selection and prioritization in CI with the goal to minimize the round-trip time between code commits and developer feedback on failed test cases. The RETECTS method uses reinforcement learning to select and prioritize test cases according to their duration, previous last execution and failure history. In a constantly changing environment, where new test cases are created and obsolete test cases are deleted, the RETECTS method learns to prioritize error-prone test cases higher under guidance of a reward function and by observing previous CI cycles. By applying RETECTS on data extracted from three industrial case studies, we show for the first time that reinforcement learning enables fruitful automatic adaptive test case selection and prioritization in CI and regression testing.

Spieker, H., Gotlieb, A., Marijan, D., & Mossige, M. (2017, July). Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (pp. 12-22).

Abstract writing advice - 2

- Don't put unexplained or undefined terms whose meanings are not well known
 - Solutions: explain them; rephrase them using plain words; not get into too much detail (without mentioning them).
- Minor details or a description of the structure of the paper are usually inappropriate
 - Acronyms, mathematical equations, abbreviations, or citations
 - Rare: if citations are needed, provide full citations.
 - Ex. Don't put [1] in the abstract

Abstract writing advice - 3

- Many recommend writing the abstract last as it provides a short and concise summary of the paper
- The key message, terms, and numbers have to be consistent across the abstract, introduction, and conclusion
- Your paper may have more than two results and conclusions, only state the most important one or two in the abstract
- Abstract must be stand-alone
 - Do not cite papers or sections of the paper in the abstract

Abstract writing advice - 4

- Abstract must be concise; do not leave readers to guess what you mean
- Avoid any grammatical errors in abstract

(This) paper fell well below my threshold. I refuse to spend time trying to understand what the author is trying to say. Besides, I really want to send a message that they can't submit garbage to us and expect us to fix it. **My rule of thumb is that if there are more than 6 grammatical errors in the abstract,** then I don't waste my time carefully reading the rest.

Source: <https://www.elsevier.com/connect/writing-a-science-paper-some-dos-and-donts>

Abstract Writing Exercise

- Find and read one paper
 - Copy the introduction section of the paper to a Google Docs
- Work in groups of two each to write an abstract for the paper you read, in 200 words
- For those groups having chosen a topic, you can write an abstract for your topic
 - Time: 20 minutes
- Compare your abstract with the original one
- Share your Google Docs with the class
- We together review and revise each abstract

Organizing (conference paper)

- Title (1000 readers)
- Abstract (4 sentences, 100 readers)
- Introduction (1 page, 100 readers)
- The problem (1 page, 10 readers)
- My idea (2 pages, 10 readers)
- The details (5 pages, 3 readers)
- Related work (1-2 pages, 10 readers)
- Conclusions and further work (0.5 pages)

Source: "[How to write a great research paper - Seven simple suggestions](#)" by, Simon Peyton Jones, Microsoft Research, Cambridge

Introduction - 1

- An introduction can be regarded as an expanded version of the abstract
 - Should not just simply restate the abstract
 - Should not contain the same sentences from the abstract
- It describes the paper's topic, the problem being studied, references to key papers, the approach to the solution, the scope and limitations of the solution, and the outcomes.
- There needs to be enough detail to allow readers to decide whether or not they need to read further.

Introduction - 2

- Be modest, do not overclaim
 - But it is good to polish and put your work in a bigger picture
- The paper needs to justify any claims made in the introduction
- Do not over-criticize other's work
- If you want to claim some unjustified points, it is better to put them in conclusion or discussion section
- Even if so, be careful on wording
 - ✗ □ “Our approach provides a foundation for this new field.”
 - ✓ □ “We believe our approach can provide a foundation...”
 - ✓ □ “We believe our approach has a good potential for providing a foundation ...”

Introduction's Structure - 1

■ Stirewalt's 5-paragraph rule

1) **Introductory paragraph**

- Very briefly: What is the problem and why is it relevant to the audience attending *THIS CONFERENCE*?
- Moreover, why is the problem hard, and what is your solution? You must be brief here. This forces you to boil down your contribution to its bare essence and communicate it directly.

2) **Background paragraph**

- Elaborate on why the problem is hard, critically examining prior work, trying to tease out one or two central shortcomings that your solution overcomes.

Source: <http://www.cse.msu.edu/%7Echengb/Writing/intro-guidelines-stirewalt.txt>

Introduction's Structure - 2

■ Stirewalt's 5-paragraph rule (cont'd)

3) **Transition paragraph**

- What keen insight did you apply to overcome the shortcomings of other approaches?
- Structure this paragraph like a syllogism: Whereas P and $P \Rightarrow Q$, infer Q.

4) **Details paragraph**

- What technical challenges did you have to overcome and what kinds of validation did you perform?

5) **Assessment paragraph**

- Assess your results and briefly state the broadly interesting conclusions that these results support. This may only take a couple of sentences. I usually then follow these sentences by an optional overview of the structure of the paper with interleaved section callouts.

Source: <http://www.cse.msu.edu/%7Echengb/Writing/intro-guidelines-stirewalt.txt>

Introduction's Structure - 3

The Stanford InfoLab's patented five-point structure:

1. *What is the problem?*
2. *Why is it interesting and important?*
3. *Why is it hard?* (e.g., why do naive approaches fail?)
4. *Why hasn't it been solved before?* (or, what's wrong with previous proposed solutions? How does mine differ?)
5. *What are the key components of my approach and results?* Also include any specific limitations.

Source: <https://cs.stanford.edu/people/widom/paper-writing.html>

Jones' Structure

- One page
 - Describe the problem
 - May give an example
 - State your contributions
 - ... and that is all

1 Introduction

There are two basic ways to implement function application in a higher-order language, when the function is unknown: the *push/enter* model or the *eval/apply* model [11]. To illustrate the difference, consider the higher-order function **zipWith**, which zips together two lists, using a function **k** to combine corresponding list elements:

```
zipWith :: (a->b->c) -> [a] -> [b] -> [c]
zipWith k []      []      = []
zipWith k (x:xs) (y:ys) = k x y : zipWith xs ys
```

Here **k** is an *unknown function*, passed as an argument; global flow analysis aside, the compiler does not know what function **k** is bound to. How should the compiler deal with the call **k x y** in the body of **zipWith**? It can't blithely apply **k** to two arguments, because **k** might in reality take just one argument and compute for a while before returning a function that consumes the next argument; or **k** might take three arguments, so that the result of the **zipWith** is a list of functions.

Use an example to introduce the problem

Source: "How to write a great research paper - Seven simple suggestions" by Simon Peyton Jones, Microsoft Research, Cambridge
<http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm>

Molehills not Mountains

Example: “Computer programs often have bugs. It is very important to eliminate these bugs [1,2]. Many researchers have tried [3,4,5,6]. It really is very important.”

Yawn!

Example: “Consider this program, which has an interesting bug. <brief description>. We will show an automatic technique for identifying and removing such bugs”

Cool!

Source: “[How to write a great research paper - Seven simple suggestions](http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm)” by
Simon Peyton Jones, Microsoft Research, Cambridge
<http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm>

State Your Contributions - 1

- Write the list of contributions first
- The list of contributions drives the entire paper: the paper substantiates the claims you have made
- Reader thinks “gosh, if they can really deliver this, that’s be exciting; I’d better read on”

Source: “[How to write a great research paper - Seven simple suggestions](http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm)” by
Simon Peyton Jones, Microsoft Research, Cambridge
<http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm>

State Your Contributions - 2

- You can state like the following
 - “The paper makes the following main contributions: + **bulleted items**”
 - “The study offers several main contributions: + **bulleted items**”
 - It is good for readers to spot out main contributions

This study offers several main contributions:

- Empirically investigating fixed-size moving windows to determine training data for COCOMO calibration. To the best of our knowledge, this is the first study splitting the data set chronologically and considering timing information for COCOMO calibration.
- Presenting two conditions for the use of fixed-size moving windows: the data set exhibits changes in software development practices over time and the model does not fully explain such changes.
- Adding empirical evidence to support the use of fixed-size moving windows of the most recently completed projects for calibrating COCOMO when the two conditions above are met. This study investigates the model and data sets that have far more effort influencing factors than the previous studies on fixed-size moving windows (Lokan and Mendes, 2009; Lokan and Mendes, 2016).
- Presenting empirical evidence in favor of using small fixed-size windows of the most recently completed projects when the two conditions are met. This evidence is helpful in practice given the lack of historical effort and size data.
- Showing that timing information existing in data affects estimation accuracy of COCOMO even although it uses many cost drivers to account for the variation in effort.

Contributions should be refutable

No!

We describe the WizWoz system. It is really cool.

We study its properties

We have used WizWoz in practice

Yes!

We give the syntax and semantics of a language that supports concurrent processes (Section 3). Its innovative features are...

We prove that the type system is sound, and that type checking is decidable (Section 4)

We have built a GUI toolkit in WizWoz, and used it to implement a text editor (Section 5). The result is half the length of the Java version.

Source: “[How to write a great research paper - Seven simple suggestions](http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm)” by
Simon Peyton Jones, Microsoft Research, Cambridge
<http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm>

Evidence

- Your introduction makes claims
- The body of the paper provides **evidence to support each claim**
- Check each claim in the introduction, identify the evidence, and forward-reference it from the claim
- “Evidence” can be: analysis and comparison, theorems, measurements, case studies

Source: “[How to write a great research paper - Seven simple suggestions](http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm)” by
Simon Peyton Jones, Microsoft Research, Cambridge
<http://research.microsoft.com/en-us/um/people/simonpj/papers/giving-a-talk/giving-a-talk.htm>

Exercise 1

- Read one paper published in good conferences
- Identify each section name and its purpose
- Suggest an outline for your paper

Exercise 2 - Analyze the introduction section

- Work on the same paper chosen in the exercise above
- Identify sentences concerning the following in the intro section
 - Context, background
 - Problem
 - Why the problem is hard and important
 - Previous work addressing the problem
 - Solution/method
 - Evaluation
 - Results
 - Claims
 - Contributions
- Is there anything missing from the intro section that prevents readers from understanding it?